

Spectroscopic Data Reduction Part 2: Wavelength Calibration

Camila Pérez Angulo
Universidad Distrital Francisco José de Caldas
Facultad de Ciencias Matemáticas y Naturales
Programa de Física
November 28, 2025

1 Spectroscopic Data Reduction Part 2: Wavelength Calibration

This notebook assumes you've completed the Spectroscopic Trace process (see [Part 1](#)) and have a trace model handy.

2 Reducción de Datos Espectroscópicos Parte 2: Calibración en Longitud de Onda

Resumen: En este cuaderno continuamos con la calibración espectral (wavelength calibration). Vamos a: - cargar imágenes de espectros y convertirlas a vectores de intensidad vs. píxel, - identificar picos (líneas espectrales) en las imágenes de referencia (He, Ne, Kr, Hg, ...), - buscar las longitudes de onda conocidas asociadas a esos picos (usando catálogos o la base de datos NIST), - ajustar un polinomio que convierta píxeles $\rightarrow$ longitud de onda, - aplicar esa calibración a espectros observacionales para poder medir características (líneas, anchos, etc.).

Autores y referencia del tutorial original: Adam Ginsburg, Kelle Cruz, Lia Corrales, Jonathan Sick, Adrian Price-Whelan

Requerimientos (traducción y explicación):

Se lee `requirements.txt` y muestra los paquetes necesarios para ejecutar el cuaderno. - Asegúrate de tener instalados los paquetes listados (por ejemplo: `astropy`, `numpy`, `matplotlib`, `scipy`, `photutils`, `specutils`, `PIL/Pillow`). - Si falta algún paquete, instálalo con `pip install -r requirements.txt` o equivalente en tu entorno. - Si trabajas en un entorno virtual, confirma que Jupyter/Notebook esté usando ese mismo entorno (kernel).

```
[1]: with open('requirements.txt') as f:
      print(f"Required packages for this notebook:\n{f.read()}")
```

Required packages for this notebook:

```
astropy
astroquery>=0.4.8.dev9474 # 2024-09-24 pinned for Gaia column capitalization
issue
IPython
numpy
pillow
```

matplotlib

Imports — explicación paso a paso y traducción:

La celda de imports carga librerías necesarias. Resumen de las más importantes:

- PIL.Image (Pillow): para abrir imágenes de espectros (jpeg, png).
- numpy: manejo de arrays y cálculos numéricos.
- matplotlib.pyplot: para graficar espectros, imágenes, perfiles.
- astropy.units y astropy.modeling.polynomial: herramientas astronómicas y modelos polinómicos (útiles para la calibración).
- scipy.signal (p. ej. find_peaks): para detectar picos en el espectro.
- photutils o specutils si está presente: rutinas específicas para fotometría/apertura/line finding.

```
[2]: from PIL import Image as PILImage
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['image.origin'] = 'lower' # we want to show images, not matrices,
↳so we set the origin to the lower-left
plt.style.use('dark_background') # Optional configuration: if run, this will
↳look nice on dark background notebooks
```

```
[3]: from astropy import units as u
from astropy.modeling.polynomial import Polynomial1D
from astropy.modeling.models import Gaussian1D, Linear1D
from astropy.modeling.fitting import LinearLSQFitter
from IPython.display import Image
# astroquery provides an interface to the NIST atomic line database
from astroquery.nist import Nist
```

```
[ ]: # Cell 1: Imports y configuración
import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from scipy.signal import find_peaks, savgol_filter
from scipy.optimize import curve_fit
import pandas as pd

# Mostrar figuras inline en Jupyter
%matplotlib inline
plt.rcParams['figure.figsize'] = (10,4)
```

Explicación de la visualización de la imagen:

- Se carga la imagen (por ejemplo: imagen del espectro de Ne, He, Kr).

- Se muestra con `plt.imshow(...)` para inspeccionar visualmente la distribución espacial de la luz.
- Objetivo: localizar la franja donde está el espectro (usualmente una franja horizontal/vertical), verificar rotación, saturación o artefactos.

Paso a paso: 1. Mira la orientación (¿el espectro está horizontal o vertical?). Si no está alineado, podría ser necesario rotar o recortar la imagen. 2. Localiza la franja donde la intensidad varía (esa es la dispersión). 3. Si hay líneas verticales brillantes y fondo estable: es buena señal para extracción.

```
[4]: hg_filename = "Mercurio.bmp"
     kr_filename = "Kripton.bmp"
     ne_filename = "Neon.bmp"
     he_filename = "Helio.bmp"
```

`img = PILImage.open('file.jpg')` → abre el archivo de imagen.

`arr = np.array(img)` → convierte la imagen a un array NumPy (shape: filas x columnas x canales).

`plt.imshow(arr, origin='lower')` → muestra la imagen para inspección visual.

Explicación paso a paso:

Abre la imagen del espectro (puede ser foto del foco del espectrógrafo o captura del detector).

Convierte la imagen a array para poder manipular píxeles numéricamente.

Muestra la imagen para verificar orientación, saturación, y la presencia de la franja espectral.

```
[5]: hg_image = np.array(PILImage.open(hg_filename))
     kr_image = np.array(PILImage.open(kr_filename))
     ne_image = np.array(PILImage.open(ne_filename))
     he_image = np.array(PILImage.open(he_filename))
```

Convertir JPEG a BMP

Se convierten las imágenes “He.jpeg”, “Kr.jpeg”, “Hg.jpeg”, “Ne.jpeg” al formato bmp

```
[ ]: # Cell 2: Convertir JPEG a BMP (opcional)
     jpg_files = ["He.jpeg", "Kr.jpeg", "Hg.jpeg", "Ne.jpeg"]
     out_format = "bmp" # o 'png'

     for fname in jpg_files:
         if not os.path.exists(fname):
             print(f"Archivo no encontrado: {fname}")
             continue
         base = Path(fname).stem
         outname = f"{base}.{out_format}"
         if not os.path.exists(outname):
             img = Image.open(fname).convert('RGB')
             img.save(outname)
             print(f"Guardado: {outname}")
```

```
else:
    print(f"Ya existe: {outname}")
```

Se carga la imagen y se convierte a 1D:

Como las imágenes poseen varios colores se convierten a arreglos unidimensionales para poder procesarlas

```
[6]: # Cell 3: Funciones auxiliares para cargar imagen y extraer perfil espectral
def load_image_gray(path):
    img = Image.open(path).convert('L')
    arr = np.array(img).astype(float)
    arr = (arr - arr.min()) / (arr.max() - arr.min() + 1e-12)
    return arr

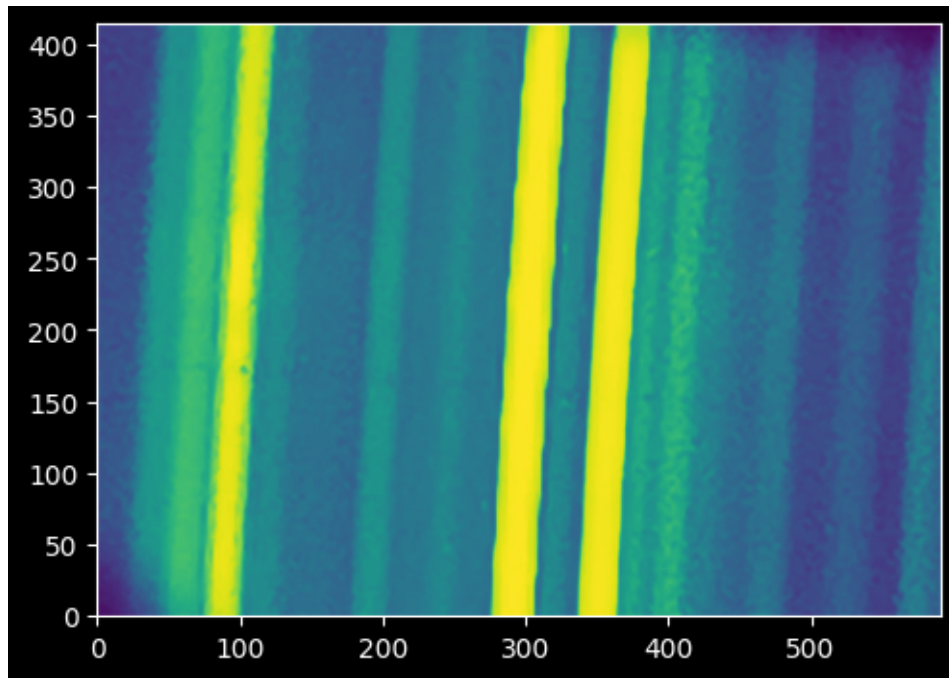
def spectral_profile(img_arr, axis=0, crop=None):
    if crop is not None:
        r0,r1,c0,c1 = crop
        img_arr = img_arr[r0:r1, c0:c1]
    return img_arr.sum(axis=axis)

def smooth_profile(profile, window=31, poly=3):
    if len(profile) < window:
        window = max(3, (len(profile)//2)*2+1)
    return savgol_filter(profile, window, poly)
```

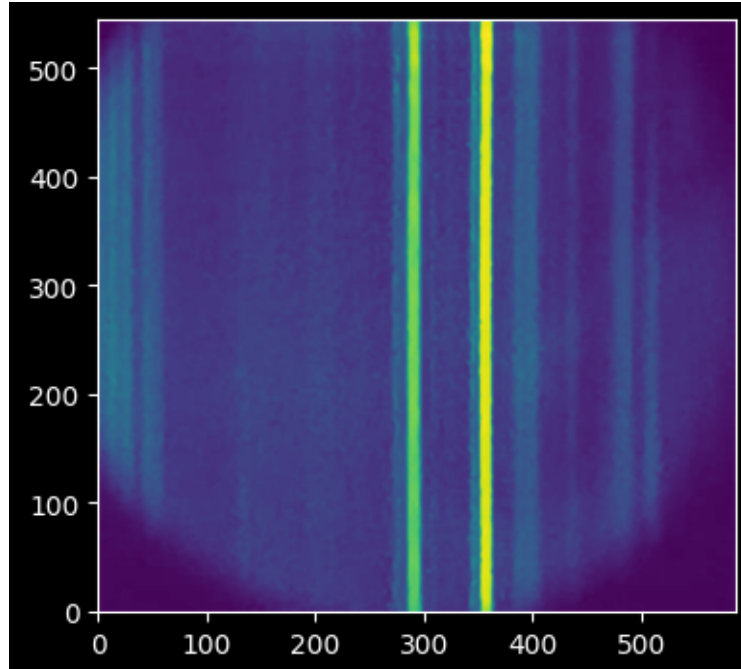
```
[8]: hg_gray = load_image_gray(hg_filename)
kr_gray = load_image_gray(kr_filename)
ne_gray = load_image_gray(ne_filename)
he_gray = load_image_gray(he_filename)
```

Impresión de las imágenes unidimensionales

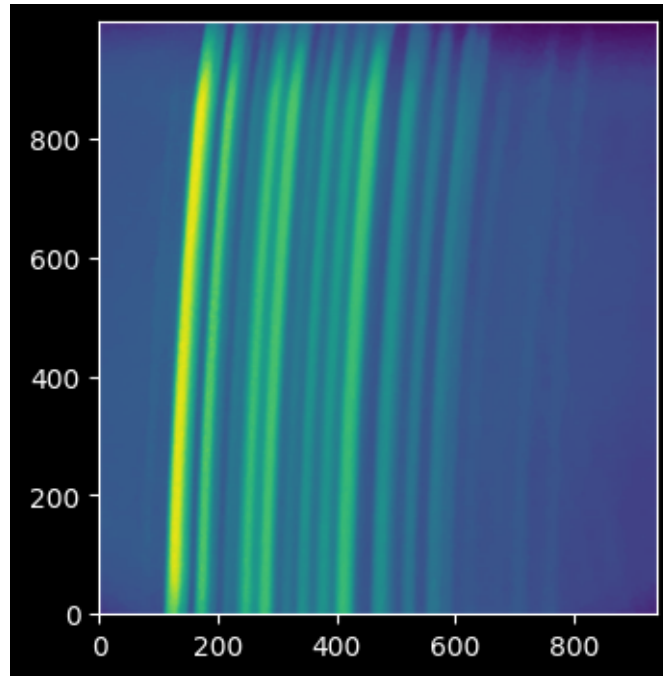
```
[9]: plt.imshow(hg_gray);
```



```
[10]: plt.imshow(kr_gray);
```

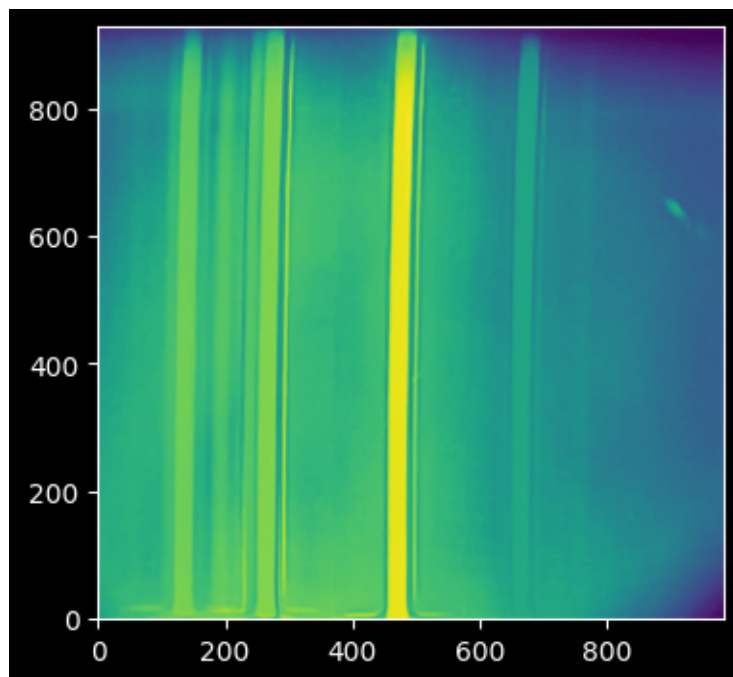


```
[11]: plt.imshow(ne_gray);
```



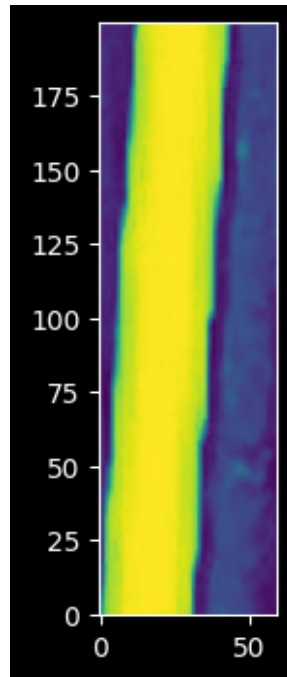
```
[12]: plt.imshow(he_gray)
```

```
[12]: <matplotlib.image.AxesImage at 0x7f2f39e0fc10>
```



Podemos observar únicamente una línea espectral haciendo zoom

```
[13]: plt.imshow(hg_gray[100:300,280:340]);
```



Nosotros recreamos nuestro modelo de traza del [Tutorial de Trazado Espectroscópico](#) usando los modelos ajustados.

Podríamos haber usado la traza empírica directamente, lo cual podría resultar en características de ruido ligeramente mejoradas, pero por simplicidad —y para hacer que ambos cuadernos sean utilizables de manera independiente— usamos aquí los modelos polinómicos y Gaussianos ajustados.

```
[89]: trace_model = Polynomial1D(degree=3, c0=276.03448116, c1=0.0091462, c2=0.  
    ↪00002231, c3=-0.00000001)  
trace_profile_model = Gaussian1D(amplitude=123.84846797, mean=0.17719819,   
    ↪stddev=5.10872134)  
xaxis = np.arange(hg_gray.shape[1])  
trace_center = trace_model(xaxis)  
npixels_to_cut=15  
yaxis = np.arange(-npixels_to_cut, npixels_to_cut)  
model_trace_profile = trace_profile_model(yaxis)
```

```
[90]: xaxis_hg = np.arange(hg_gray.shape[1])  
trace_center_hg = trace_model(xaxis_hg)
```

```
[91]: xaxis_ne = np.arange(ne_gray.shape[1])  
trace_center_ne = trace_model(xaxis_ne)
```

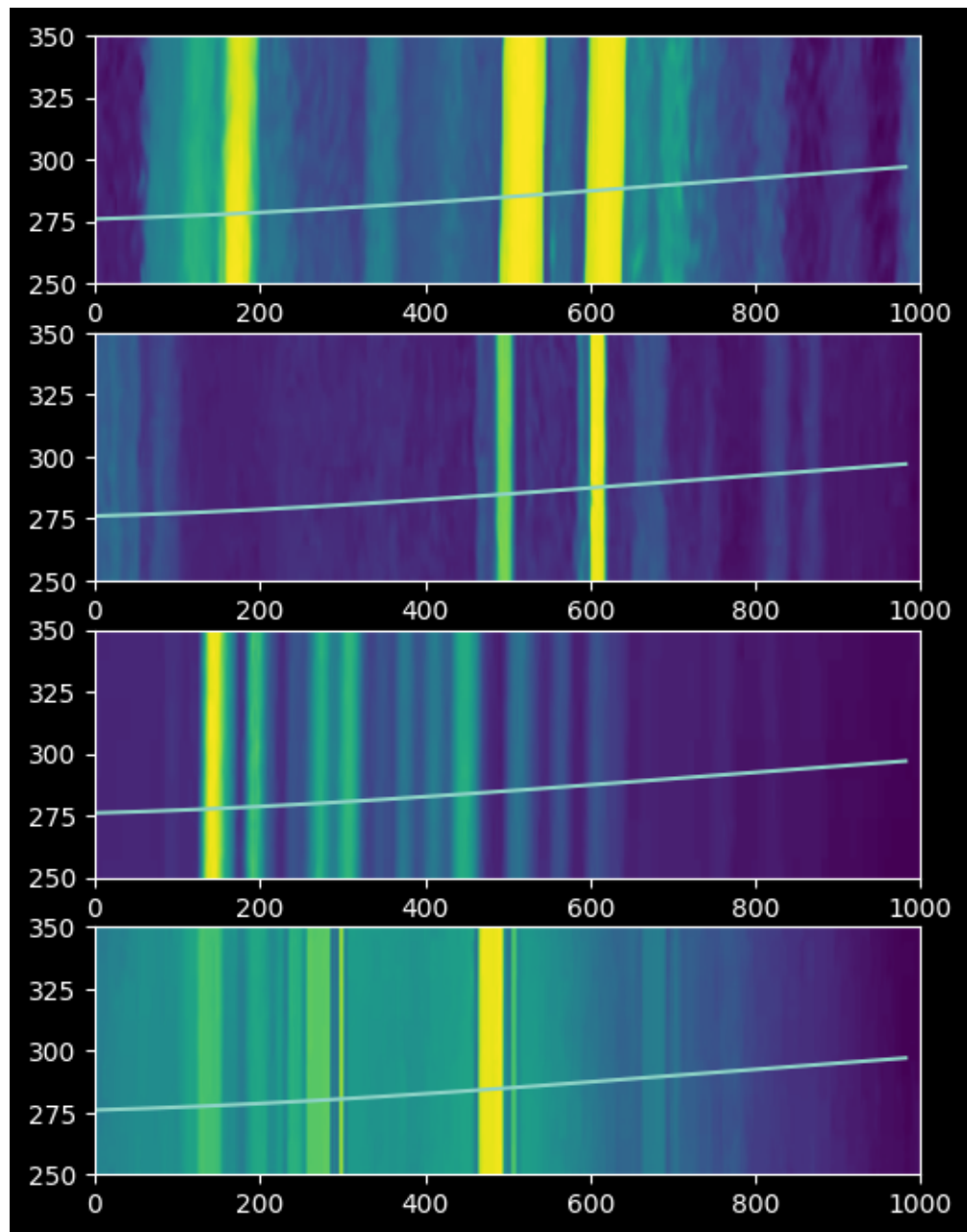
```
[92]: xaxis_kr = np.arange(kr_gray.shape[1])
      trace_center_kr = trace_model(xaxis_kr)
```

```
[93]: xaxis_he = np.arange(he_gray.shape[1])
      trace_center_he = trace_model(xaxis_he)
```

Luego, usamos la traza para extraer espectros de cada uno de los tres espectros de las lámparas de calibración.

Primero verificamos que las trazas se vean aceptables, ajustamos los límites de los ejes x y y para poder observar el espectro y las trazas de la mejor manera posible

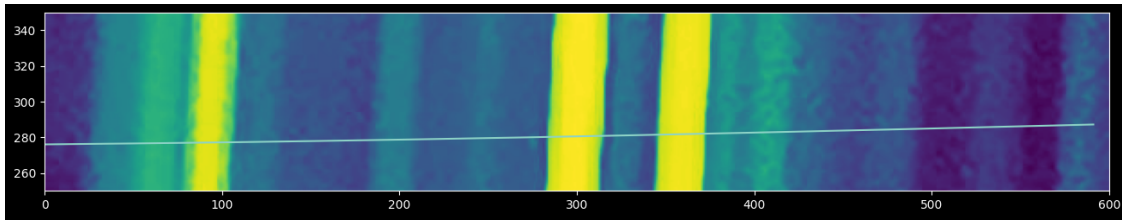
```
[98]: plt.figure(figsize=(16,8))
      ax1 = plt.subplot(4,1,1)
      ax1.imshow(hg_gray[250:300,:], extent=[0,1000,250,350])
      ax1.plot(xaxis_he, trace_center_he)
      ax1.set_aspect(3)
      ax2 = plt.subplot(4,1,2)
      ax2.imshow(kr_gray[250:300,:], extent=[0,1000,250,350])
      ax2.plot(xaxis_he, trace_center_he)
      ax2.set_aspect(3)
      ax3 = plt.subplot(4,1,3)
      ax3.imshow(ne_gray[250:300,:], extent=[0,1000,250,350])
      ax3.plot(xaxis_he, trace_center_he)
      ax3.set_aspect(3)
      ax4 = plt.subplot(4,1,4)
      ax4.imshow(he_gray[250:300,:], extent=[0,1000,250,350])
      ax4.plot(xaxis_he, trace_center_he)
      ax4.set_aspect(3)
```



Mostramos individualmente cada espectro con su respectiva traza, para cada uno de los elementos empleados:

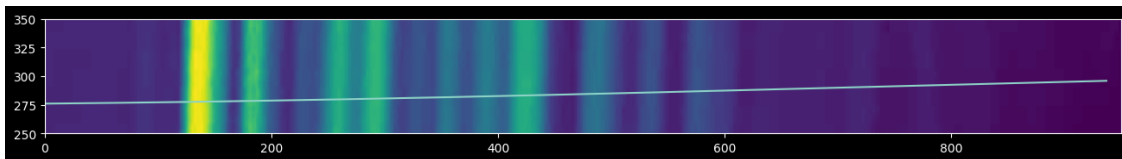
```
[99]: plt.figure(figsize=(16,8))
      ax1 = plt.subplot(1,1,1)
      ax1.imshow(hg_gray[50:150,:], extent=[0, 600,250,350])
      ax1.plot(xaxis_hg, trace_center_hg)
```

```
[99]: [<matplotlib.lines.Line2D at 0x7f2f3421ffd0>]
```



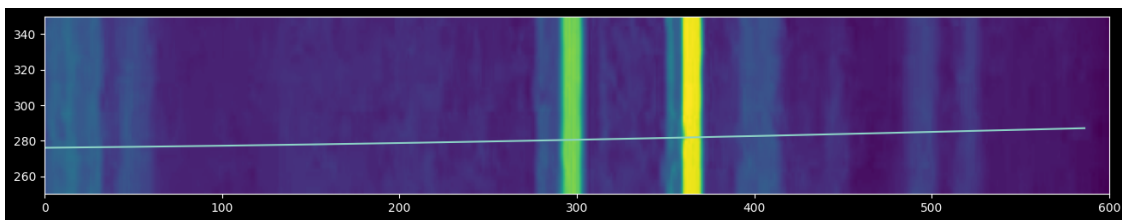
```
[100]: plt.figure(figsize=(16,8))
ax1 = plt.subplot(1,1,1)
ax1.imshow(ne_gray[250:300,:], extent=[0, 950,250,350])
ax1.plot(xaxis_ne, trace_center_ne)
```

[100]: [<matplotlib.lines.Line2D at 0x7f2f342e0b90>]



```
[101]: plt.figure(figsize=(16,8))
ax1 = plt.subplot(1,1,1)
ax1.imshow(kr_gray[250:300,:], extent=[0, 600,250,350])
ax1.plot(xaxis_kr, trace_center_kr)
```

[101]: [<matplotlib.lines.Line2D at 0x7f2f34172410>]



```
[102]: plt.figure(figsize=(16,8))
ax1 = plt.subplot(1,1,1)
ax1.imshow(he_gray[250:300,:], extent=[0, 1000,250,350])
ax1.plot(xaxis_he, trace_center_he)
```

[102]: [<matplotlib.lines.Line2D at 0x7f2f34198550>]



En las siguientes tres celdas, extraemos el espectro promedio ponderado por el perfil de la traza de cada uno de los diferentes elementos que se tienen

`npixels_to_cut` establece una región recortada alrededor de la traza, similar a lo que se muestra en las figuras anteriores. El `model_trace_profile`, derivado anteriormente, es el perfil de transmisión que medimos usando la traza estelar.

```
[103]: hg_spectrum = np.array([np.average(hg_gray[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center_hg, xaxis_hg)])
```

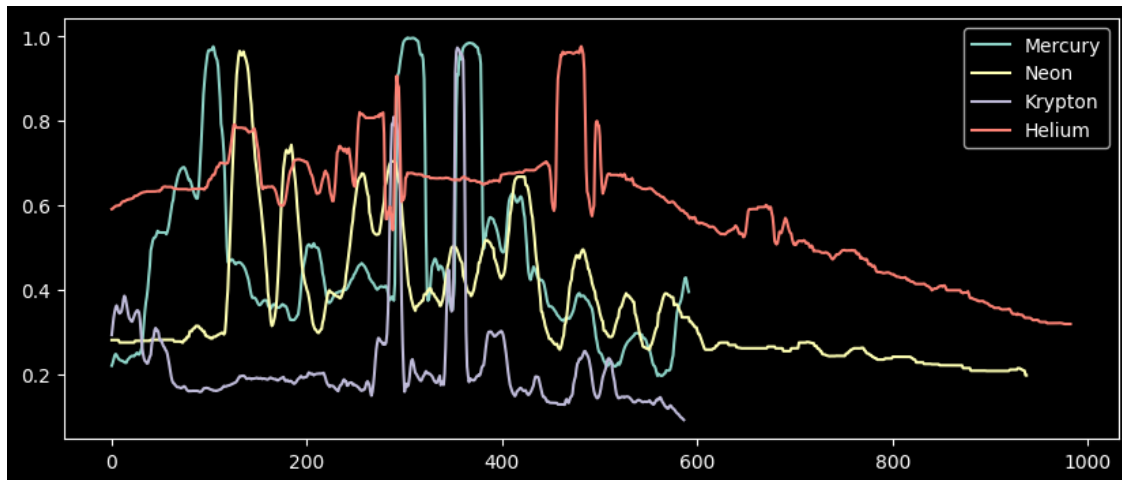
```
[104]: ne_spectrum = np.array([np.average(ne_gray[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center_ne, xaxis_ne)])
```

```
[105]: kr_spectrum = np.array([np.average(kr_gray[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center_kr, xaxis_kr)])
```

```
[106]: he_spectrum = np.array([np.average(he_gray[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center_he, xaxis_he)])
```

Se observan los espectros de los elementos

```
[107]: plt.plot(xaxis_hg, hg_spectrum, label='Mercury')
plt.plot(xaxis_ne, ne_spectrum, label='Neon')
plt.plot(xaxis_kr, kr_spectrum, label='Krypton')
plt.plot(xaxis_he, he_spectrum, label='Helium')
plt.legend(loc='best');
```



```
[108]: from IPython.display import Image, display
import requests
```

```
Image("https://upload.wikimedia.org/wikipedia/commons/2/29/Mercury_Spectra.jpg")
```

```
Image("https://upload.wikimedia.org/wikipedia/commons/9/99/Neon_spectra.jpg")
```

```
Image("https://upload.wikimedia.org/wikipedia/commons/a/a6/Krypton_Spectrum.jpg")
```

```
[16]: url = "https://upload.wikimedia.org/wikipedia/commons/2/29/Mercury_Spectra.jpg"
headers = {"User-Agent": "Mozilla/5.0"}

r = requests.get(url, headers=headers, timeout=15)
r.raise_for_status()          # lanza error si no es 200
display(Image(data=r.content)) # muestra el binario directamente
```



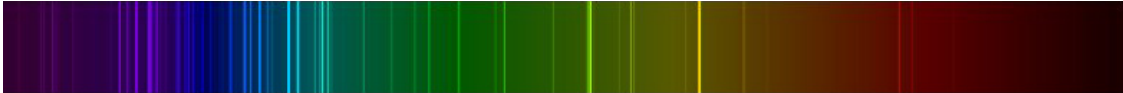
```
[17]: url = "https://upload.wikimedia.org/wikipedia/commons/9/99/Neon_spectra.jpg"
headers = {"User-Agent": "Mozilla/5.0"}

r = requests.get(url, headers=headers, timeout=15)
r.raise_for_status()          # lanza error si no es 200
display(Image(data=r.content)) # muestra el binario directamente
```



```
[18]: url = "https://upload.wikimedia.org/wikipedia/commons/a/a6/Krypton_Spectrum.jpg"
headers = {"User-Agent": "Mozilla/5.0"}

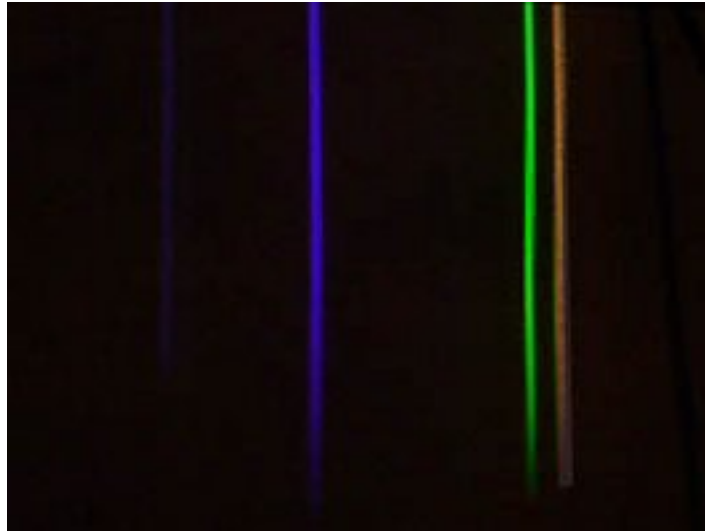
r = requests.get(url, headers=headers, timeout=15)
r.raise_for_status()          # lanza error si no es 200
display(Image(data=r.content)) # muestra el binario directamente
```



Image("https://upload.wikimedia.org/wikipedia/commons/0/0d/HG-Spektrum_crop.jpg")

```
[19]: url = "https://upload.wikimedia.org/wikipedia/commons/0/0d/HG-Spektrum_crop.jpg"
headers = {"User-Agent": "Mozilla/5.0"}

r = requests.get(url, headers=headers, timeout=15)
r.raise_for_status()          # lanza error si no es 200
display(Image(data=r.content)) # muestra el binario directamente
```



The optical lines of Mercury are, from Wikipedia, in Angstroms:

- 4047 violet
- 4358 blue
- 5461 green
- 5782 yellow-orange

- 6500 red

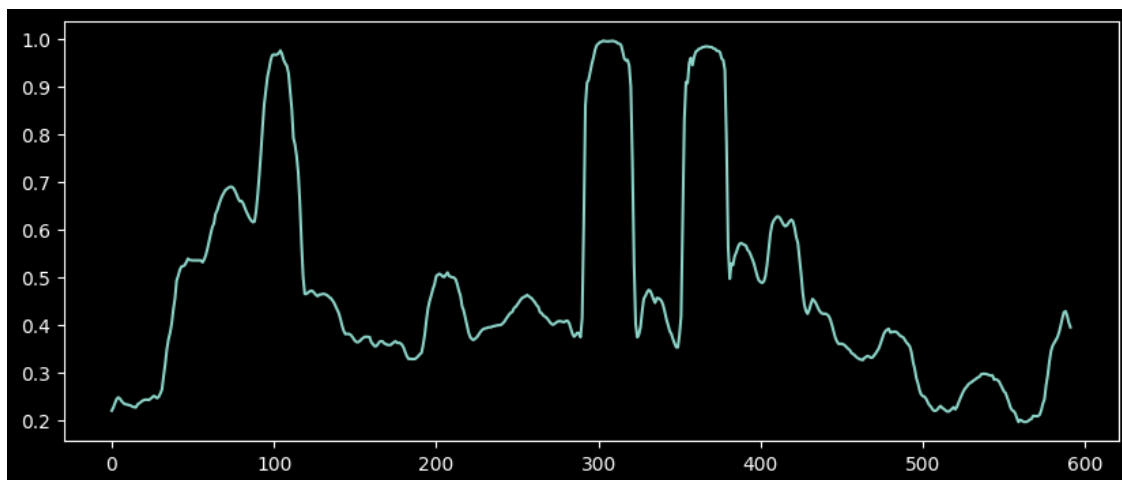
Se imprimen individualmente los espectros del Mercurio, el Neón, el Kriptón y el Helio respectivamente, se escogen los picos que van a ser comparados con los valores teóricos sacados del NIST o de alguna base de datos que nos permita saber las líneas espectrales características de cada element químico. Tenemos que tener en cuenta los pixeles en los que están ubicados estos picos que vamos a comparar.

¿Qué es el NIST?

El **NIST** (National Institute of Standards and Technology), o *Instituto Nacional de Estándares y Tecnología* de los Estados Unidos, es una institución gubernamental dedicada a desarrollar y mantener estándares de medición, constantes físicas, tecnologías de calibración y recursos científicos de referencia. En espectroscopía, el NIST es especialmente reconocido por su **base de datos de líneas atómicas**, la cual reúne longitudes de onda, transiciones electrónicas y otras propiedades fundamentales utilizadas para analizar y calibrar espectros.

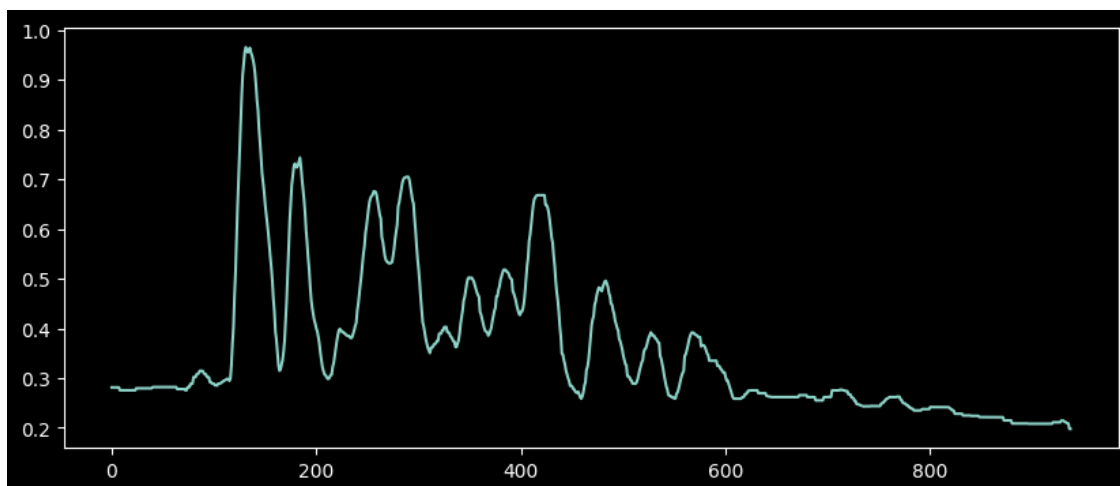
Mercurio

```
[110]: plt.plot(xaxis_hg, hg_spectrum);
```



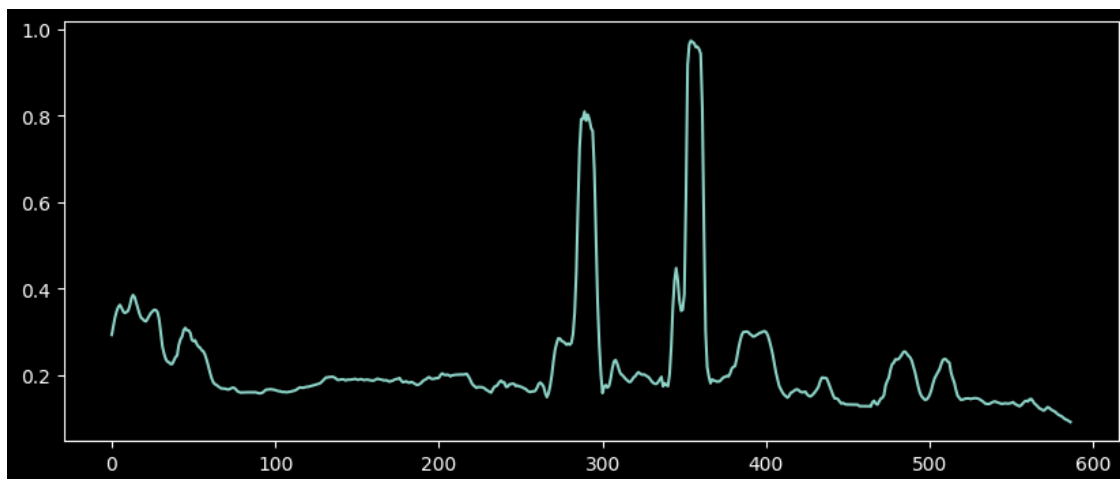
Neón

```
[111]: plt.plot(xaxis_ne, ne_spectrum);
```



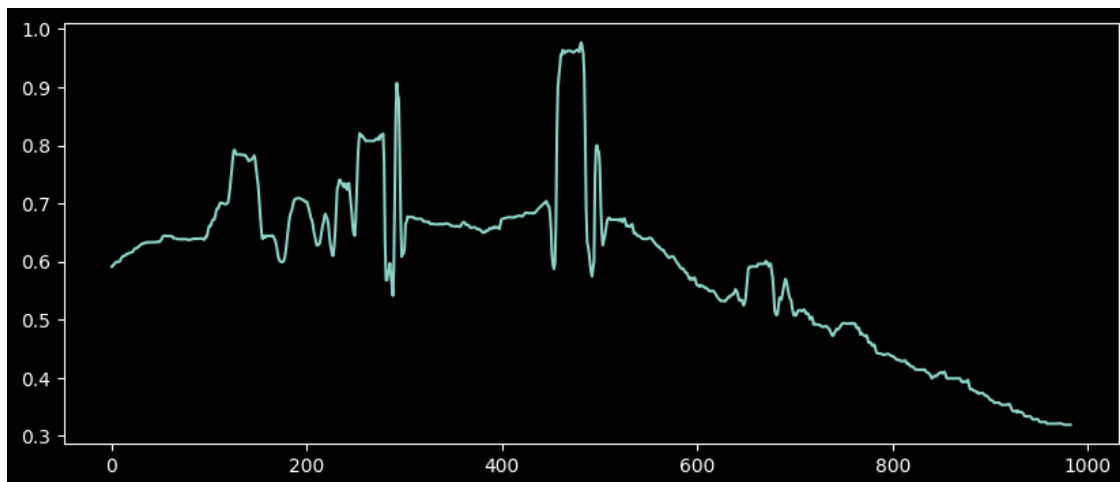
Kriptón

```
[112]: plt.plot(xaxis_kr, kr_spectrum);
```



Helio

```
[113]: plt.plot(xaxis_he, he_spectrum);
```



En este siguiente paso, tenemos que anotar qué valores (aproximados) de píxel corresponden a longitudes de onda conocidas.

Adivinar esto simplemente mirando los espectros y comparándolos con atlas espectrales es difícil y requiere experiencia.

El siguiente truco es averiguar en qué dirección aumenta la longitud de onda: ¿hacia la izquierda o hacia la derecha? El mercurio no ayuda con eso, ya que los dobletes azul-violeta y verde-amarillo están aproximadamente a la misma distancia entre sí. Resulta que, a partir de trabajo de prueba y error, la longitud de onda aumenta hacia la derecha.

En la siguiente línea de código lo que se hace es ubicar los píxeles que se van a estudiar y las longitudes de onda teóricas que se tienen para dichas líneas de emisión, tener presente que deben ser la misma cantidad (las longitudes de onda se encuentran en nm)

Mercurio

```
[221]: guessed_wavelengths_hg = [435.49, 491.23, 545.82, 567.65, 587.19, 606.73]
       guessed_xvals_hg = [100, 203, 302, 363, 408, 472]
```

2.1 Mejorando nuestras suposiciones

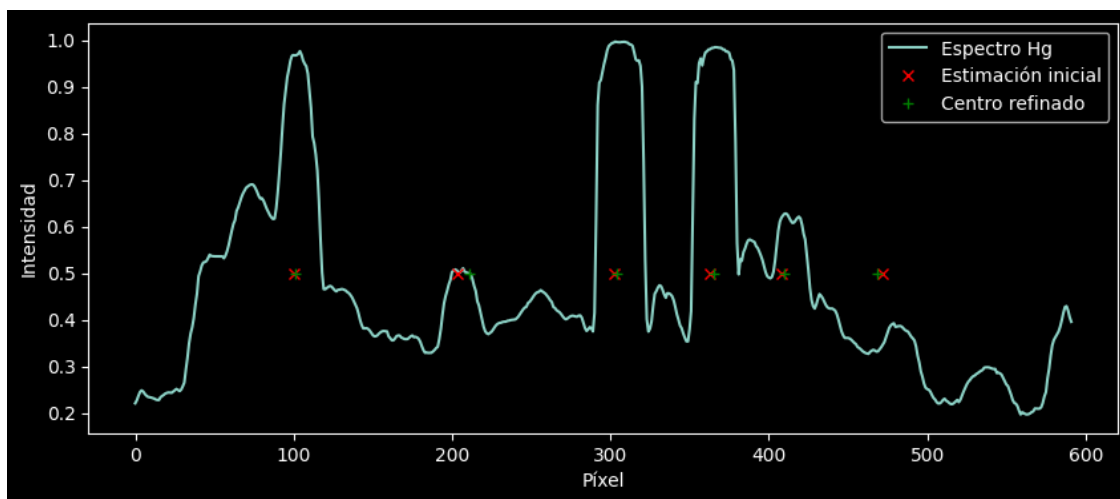
Podemos hacerlo mucho mejor al determinar los valores X de píxel tomando la coordenada ponderada por la intensidad (momento 1):

```
[222]: npixels = 15
       improved_xval_guesses_hg = [np.average(xaxis_hg[g-npixels:g+npixels],
                                             weights=hg_spectrum[g-npixels:g+npixels]
       ↪ np.median(hg_spectrum))
       for g in guessed_xvals_hg]
       improved_xval_guesses_hg
```

```
[222]: [np.float64(101.01815345563979),
        np.float64(210.8963857840367),
        np.float64(304.20276711219304),
        np.float64(365.23598908219697),
        np.float64(409.5826508526847),
        np.float64(468.24336002098846)]
```

Observamos los valores de nuestras suposiciones y los mejorados, si coinciden con los picos seleccionados se está realizando de forma óptima, si quedan muy lejos hay que cambiar el valor del pixel de nuestras suposiciones

```
[223]: plt.plot(xaxis_hg, hg_spectrum, label='Espectro Hg')
plt.plot(guesses_xvals_hg, [0.5]*6, 'rx', label='Estimación inicial')
plt.plot(improved_xval_guesses_hg, [0.5]*6, 'g+', label='Centro refinado')
plt.legend()
plt.xlabel('Píxel')
plt.ylabel('Intensidad')
plt.show()
```



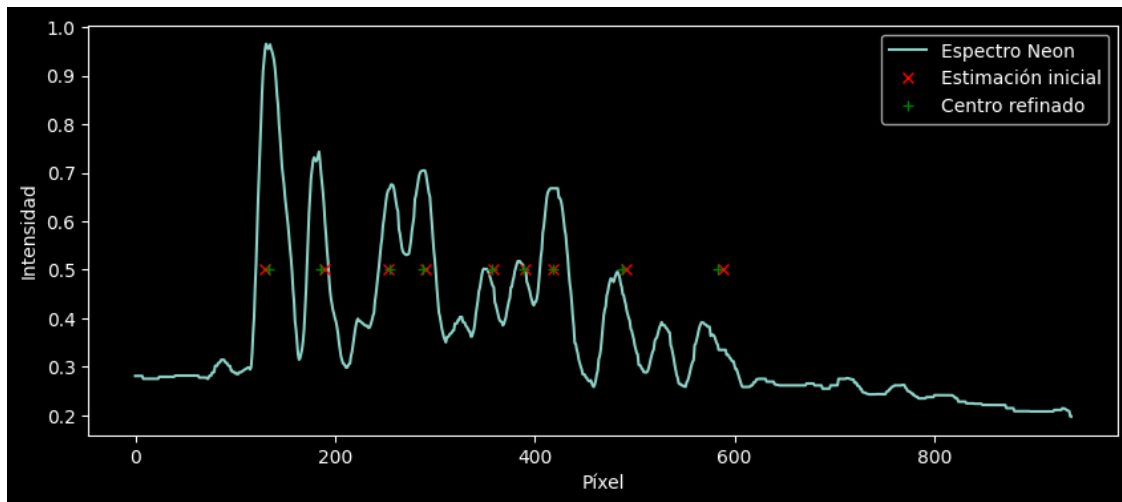
Neón

```
[153]: guessed_wavelengths_ne = [584.47, 587.36, 594.66, 597.53,
                                603.28, 607.3, 617.65, 622.24, 626.84]
guessed_xvals_ne = [130, 190, 254, 290, 359, 390, 418, 492, 588]
```

```
[154]: npixels = 15
improved_xval_guesses_ne = [np.average(xaxis_ne[g-npixels:g+npixels],
                                     weights=ne_spectrum[g-npixels:g+npixels]
                                     ↪ np.median(ne_spectrum))
                             for g in guessed_xvals_ne]
improved_xval_guesses_ne
```

```
[154]: [np.float64(133.01102911965515),
        np.float64(185.5284212469613),
        np.float64(254.7748769737537),
        np.float64(288.41034410972594),
        np.float64(356.4902213968072),
        np.float64(388.7223531978048),
        np.float64(418.2268099738853),
        np.float64(486.88728194468274),
        np.float64(582.5898134276622)]
```

```
[155]: plt.plot(xaxis_ne, ne_spectrum, label='Espectro Neon')
plt.plot(guesses_xvals_ne, [0.5]*9, 'rx', label='Estimación inicial')
plt.plot(improved_xval_guesses_ne, [0.5]*9, 'g+', label='Centro refinado')
plt.legend()
plt.xlabel('Píxel')
plt.ylabel('Intensidad')
plt.show()
```



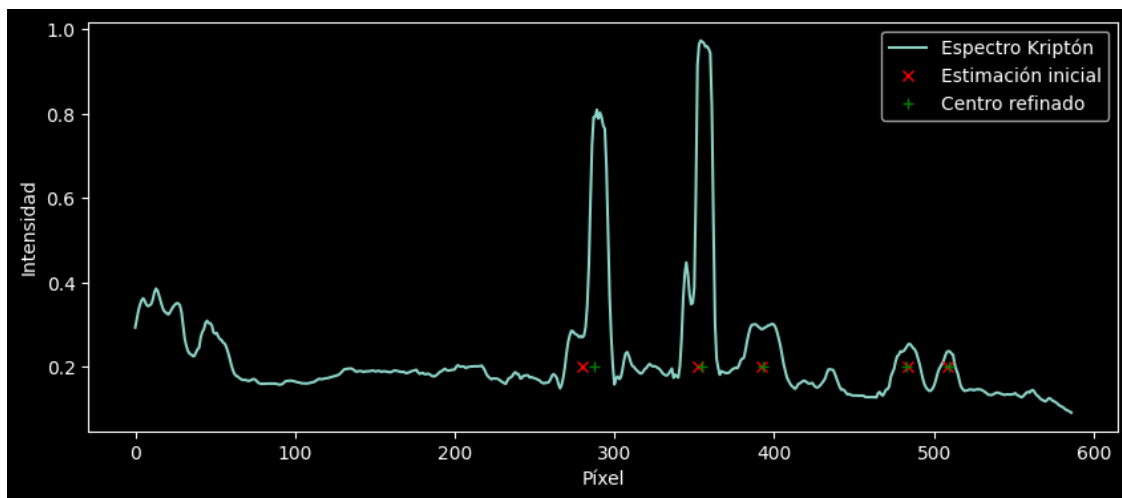
Krptón

```
[179]: guessed_wavelengths_kr = [556.73, 587.19, 605.58, 642.36, 645.8]
        guessed_xvals_kr = [280, 352, 392, 484, 509]
```

```
[180]: npixels = 15
        improved_xval_guesses_kr = [np.average(xaxis_kr[g-npixels:g+npixels],
                                                weights=kr_spectrum[g-npixels:g+npixels] -
        ↪ np.median(kr_spectrum))
                                     for g in guessed_xvals_kr]
        improved_xval_guesses_kr
```

```
[180]: [np.float64(287.53134824826685),
        np.float64(354.8258707526527),
        np.float64(392.70848061769647),
        np.float64(482.87929801092395),
        np.float64(509.05712702013045)]
```

```
[181]: plt.plot(xaxis_kr, kr_spectrum, label='Espectro Kriptón')
plt.plot(guesses_xvals_kr, [0.2]*5, 'rx', label='Estimación inicial')
plt.plot(improved_xval_guesses_kr, [0.2]*5, 'g+', label='Centro refinado')
plt.legend()
plt.xlabel('Píxel')
plt.ylabel('Intensidad')
plt.show()
```



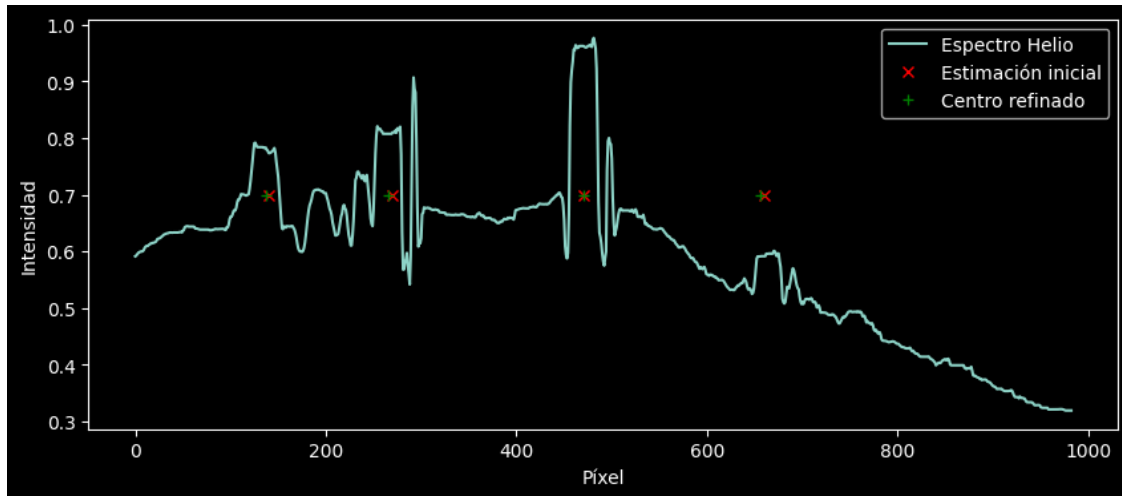
Helio

```
[199]: guessed_wavelengths_he = [446.98, 491.8, 587.19, 668.21]
        guessed_xvals_he = [140, 270, 470, 660]
```

```
[200]: npixels = 15
        improved_xval_guesses_he = [np.average(xaxis_he[g-npixels:g+npixels],
                                                weights=he_spectrum[g-npixels:g+npixels] -
        ↪ np.median(he_spectrum))
                                     for g in guessed_xvals_he]
        improved_xval_guesses_he
```

```
[200]: [np.float64(137.6654284329619),
        np.float64(266.2029225364427),
        np.float64(470.3120139965751),
        np.float64(655.9549174964578)]
```

```
[201]: plt.plot(xaxis_he, he_spectrum, label='Espectro Helio')
plt.plot(guesses_xvals_he, [0.7]*4, 'rx', label='Estimación inicial')
plt.plot(improved_xval_guesses_he, [0.7]*4, 'g+', label='Centro refinado')
plt.legend()
plt.xlabel('Píxel')
plt.ylabel('Intensidad')
plt.show()
```



```
[224]: linfitter = LinearLSQFitter()
```

Usamos un modelo `Linear1D` porque más adelante querremos usar su inversa (otros modelos no son invertibles).

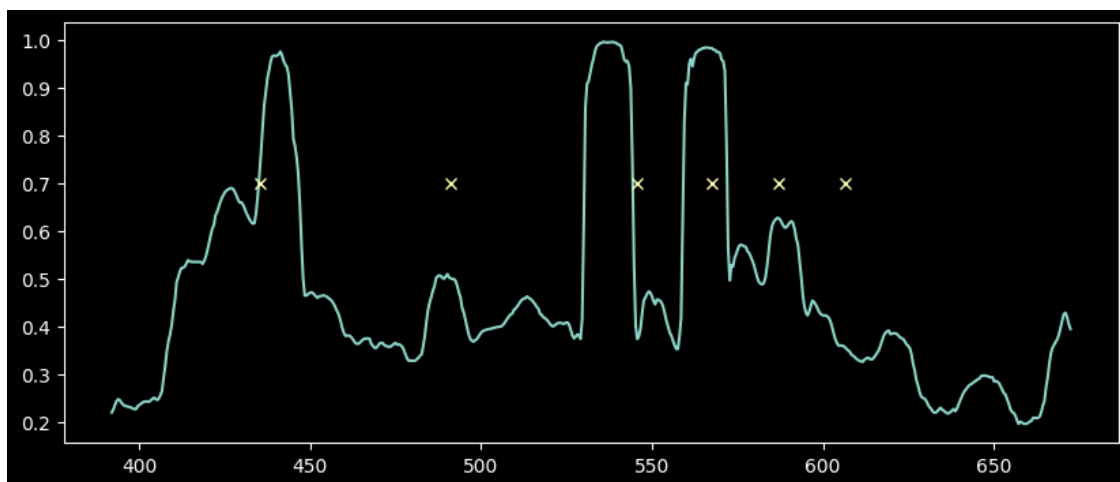
Se grafica ahora en longitudes de onda, en vez de pixeles

Mercurio

```
[231]: wlmodel = Linear1D()
linfit_wlmodel_hg = linfitter(model=wlmodel, x=improved_xval_guesses_hg,
    ↪ y=guessees_wavelengths_hg)
wavelengths_hg = linfit_wlmodel_hg(xaxis_hg) * u.nm
linfit_wlmodel_hg
```

```
[231]: <Linear1D(slope=0.47440419, intercept=392.01792493)>
```

```
[232]: plt.plot(wavelengths_hg, hg_spectrum)
plt.plot(guesses_wavelengths_hg, [0.7]*6, 'x');
```

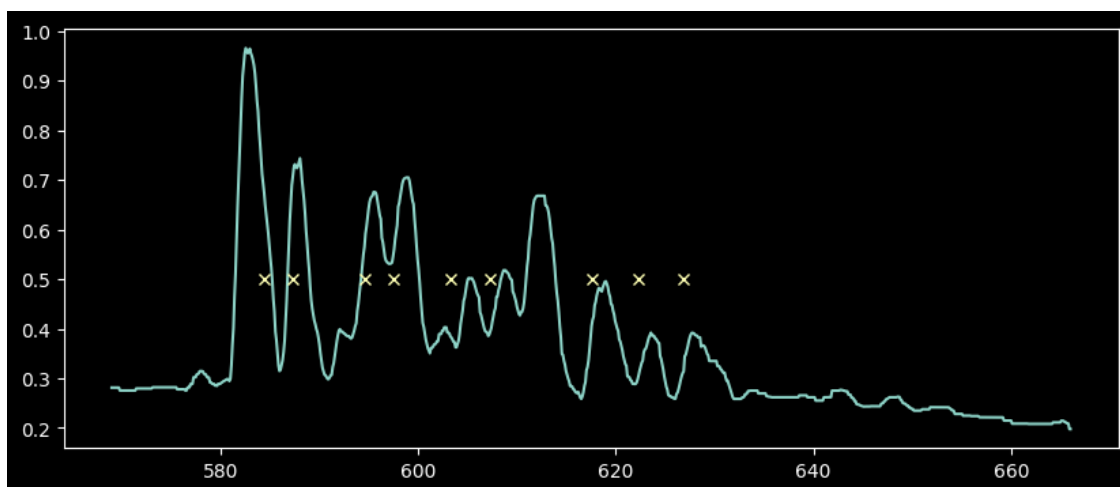


Neón

```
[234]: wlmodel = Linear1D()
linfit_wlmodel_ne = linfitter(model=wlmodel, x=improved_xval_guesses_ne, u
    ↪ y=guessed_wavelengths_ne)
wavelengths_ne = linfit_wlmodel_ne(xaxis_ne) * u.nm
linfit_wlmodel_ne
```

```
[234]: <Linear1D(slope=0.10356106, intercept=568.98285354)>
```

```
[235]: plt.plot(wavelengths_ne, ne_spectrum)
plt.plot(guessed_wavelengths_ne, [0.5]*9, 'r');
```

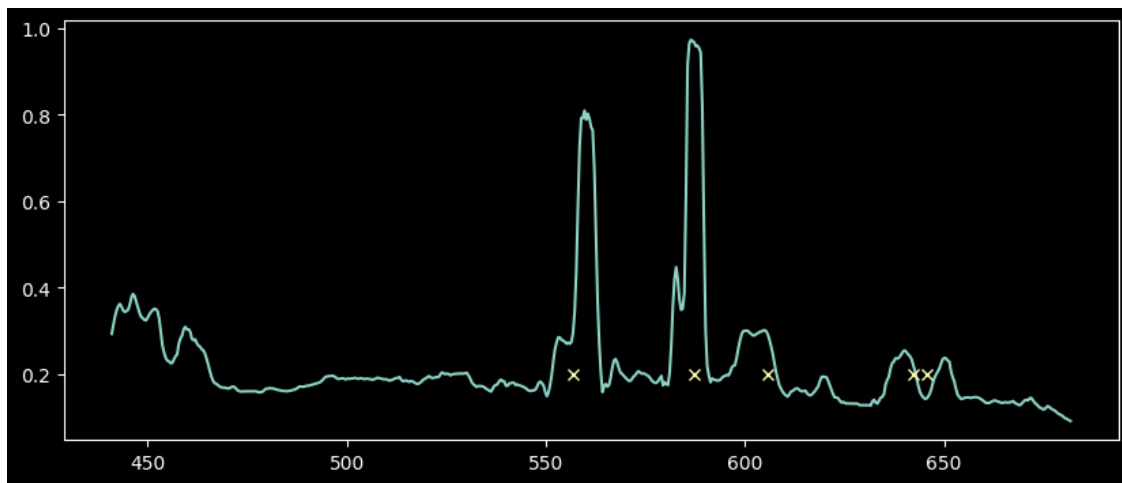


Kriptón

```
[236]: wlmodel = Linear1D()
linfit_wlmodel_kr = linfitter(model=wlmodel, x=improved_xval_guesses_kr,
    ↪y=guessed_wavelengths_kr)
wavelengths_kr = linfit_wlmodel_kr(xaxis_kr) * u.nm
linfit_wlmodel_kr
```

```
[236]: <Linear1D(slope=0.41087675, intercept=440.96239182)>
```

```
[238]: plt.plot(wavelengths_kr, kr_spectrum)
plt.plot(guessed_wavelengths_kr, [0.2]*5, 'x');
```

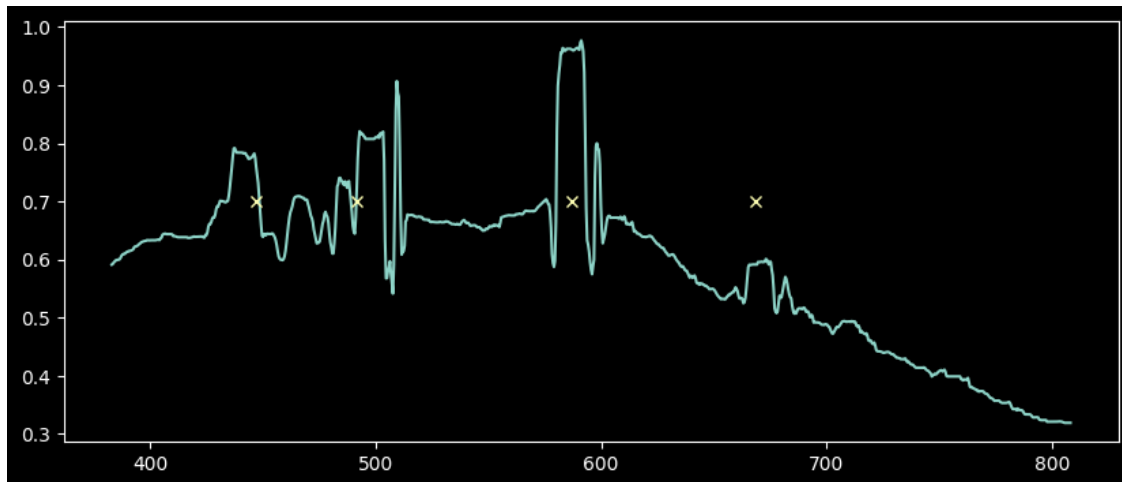


Helio

```
[239]: wlmodel = Linear1D()
linfit_wlmodel_he = linfitter(model=wlmodel, x=improved_xval_guesses_he,
    ↪y=guessed_wavelengths_he)
wavelengths_he = linfit_wlmodel_he(xaxis_he) * u.nm
linfit_wlmodel_he
```

```
[239]: <Linear1D(slope=0.43333839, intercept=382.77840874)>
```

```
[241]: plt.plot(wavelengths_he, he_spectrum)
plt.plot(guessed_wavelengths_he, [0.7]*4, 'x');
```

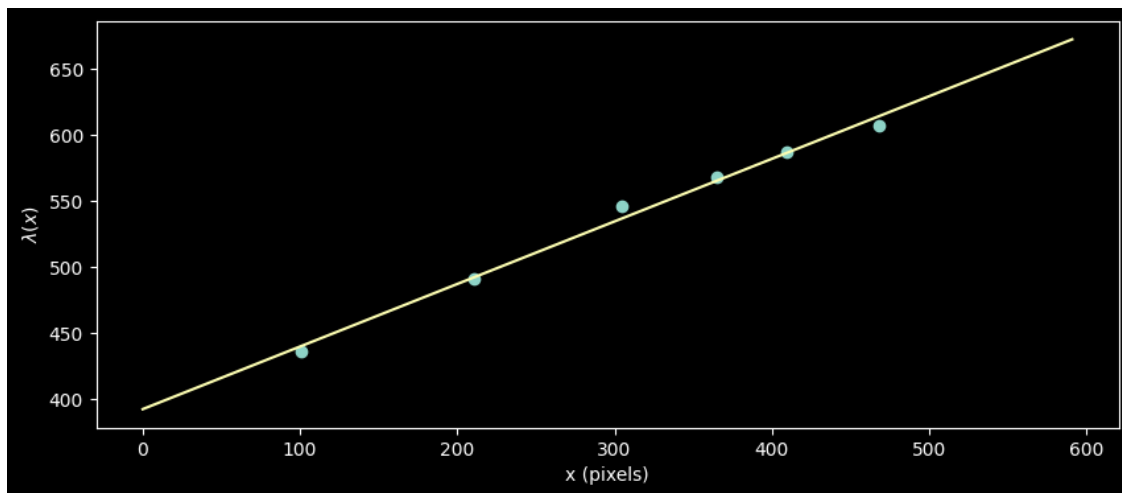


Mostramos nuestro modelo $\lambda(x)$ frente a las “suposiciones” de entrada:

Mercurio

```
[242]: plt.plot(improved_xval_guesses_hg, guessed_wavelengths_hg, 'o')
plt.plot(xaxis_hg, wavelengths_hg, '-')
plt.ylabel(r"$ \lambda(x)$")
plt.xlabel("x (pixels)")
```

```
[242]: Text(0.5, 0, 'x (pixels)')
```

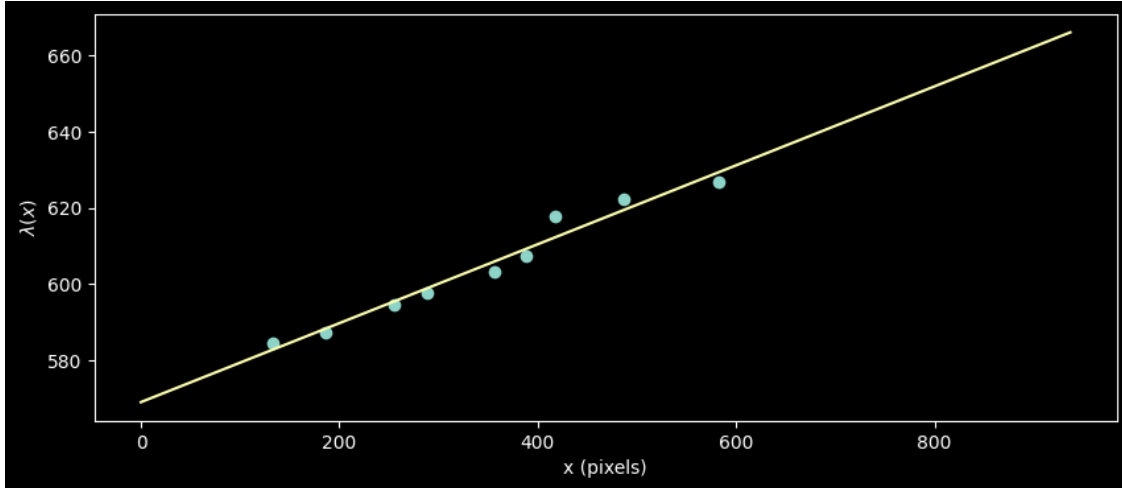


Neón

```
[243]: plt.plot(improved_xval_guesses_ne, guessed_wavelengths_ne, 'o')
plt.plot(xaxis_ne, wavelengths_ne, '-')
plt.ylabel(r"$ \lambda(x)$")
plt.xlabel("x (pixels)")
```

```
plt.ylabel(r"$ \lambda(x)$")
plt.xlabel("x (pixels)")
```

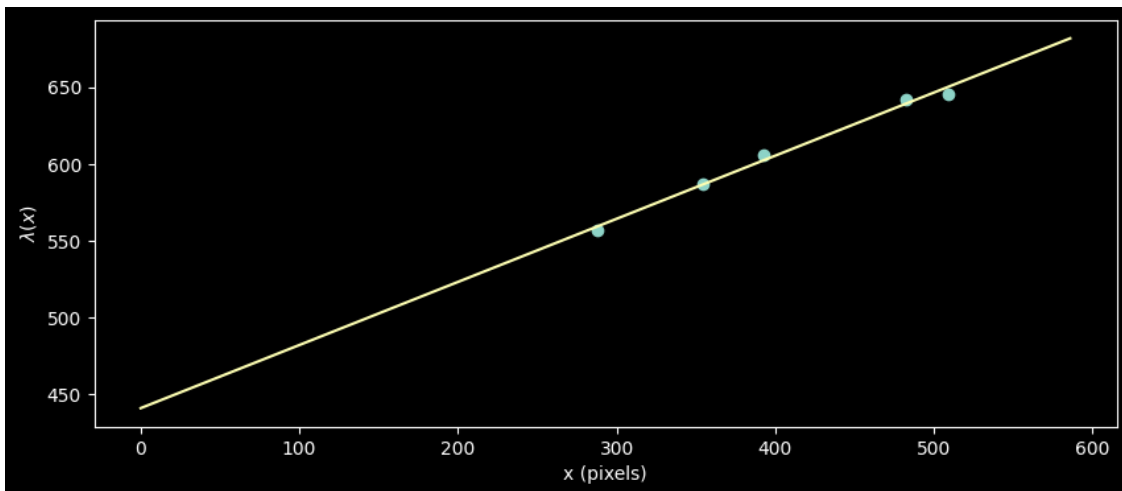
[243]: Text(0.5, 0, 'x (pixels)')



Kriptón

```
[244]: plt.plot(improved_xval_guesses_kr, guessed_wavelengths_kr, 'o')
plt.plot(xaxis_kr, wavelengths_kr, '-')
plt.ylabel(r"$ \lambda(x)$")
plt.xlabel("x (pixels)")
```

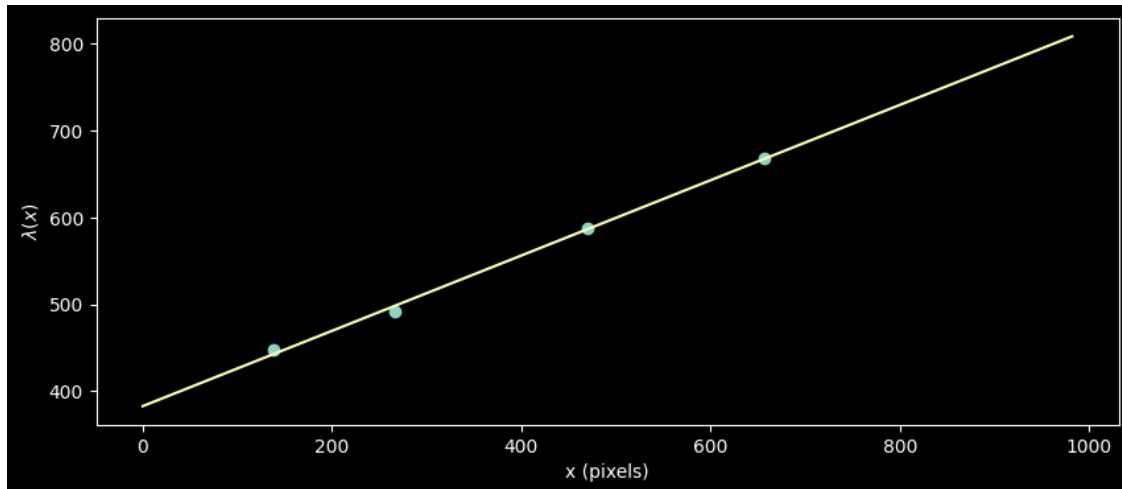
[244]: Text(0.5, 0, 'x (pixels)')



Helio

```
[245]: plt.plot(improved_xval_guesses_he, guessed_wavelengths_he, 'o')
plt.plot(xaxis_he, wavelengths_he, '-')
plt.ylabel(r"$\lambda(x)$")
plt.xlabel("x (pixels)")
```

```
[245]: Text(0.5, 0, 'x (pixels)')
```



¡En efecto, un modelo lineal ajustó excelentemente!

Podemos consultar las listas de líneas de neón, kriptón y helio para ver qué tan bien lo hicimos y quizá intentar mejorar nuestro ajuste.

NIST, el Instituto Nacional de Estándares y Tecnología, mantiene [listas de líneas atómicas](#). Sin embargo, ¡no tenemos una manera directa de saber qué transiciones de estos átomos usar! Aun así, podemos aplicar algunas heurísticas.

```
[250]: # we adopt the minimum/maximum wavelength from our linear fit
minwave_hg = wavelengths_hg.min()
maxwave_hg = wavelengths_hg.max()

minwave_ne = wavelengths_ne.min()
maxwave_ne = wavelengths_ne.max()

minwave_kr = wavelengths_kr.min()
maxwave_kr = wavelengths_kr.max()

minwave_he = wavelengths_he.min()
maxwave_he = wavelengths_he.max()

# then we search for atomic lines
# We are only interested in neutral lines, assuming the lamps are not hot enough
↳ to ionize the atoms
```

```

mercury_lines = Nist.query(minwav=minwave_hg,
                           maxwav=maxwave_hg,
                           linename='Hg I')
krypton_lines = Nist.query(minwav=minwave_kr,
                           maxwav=maxwave_kr,
                           linename='Kr I')
neon_lines = Nist.query(minwav=minwave_ne,
                        maxwav=maxwave_ne,
                        linename='Ne I')
helium_lines = Nist.query(minwav=minwave_he,
                          maxwav=maxwave_he,
                          linename='He I')

```

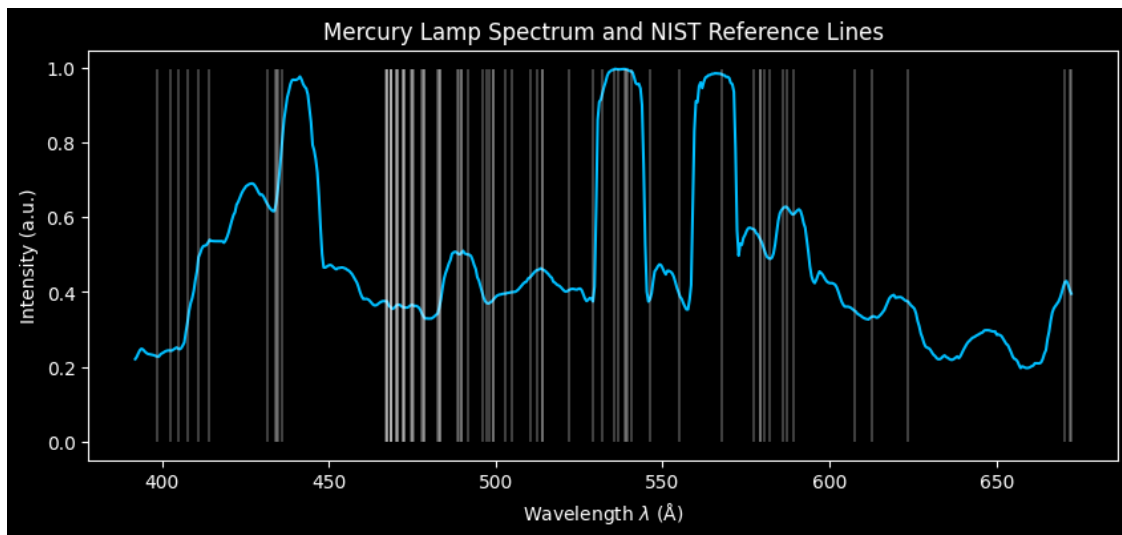
Podemos comenzar mirando lo que NIST reporta para el espectro de mercurio que ya ajustamos:
(nota que las longitudes de onda son longitudes de onda *en aire*, no en vacío)

Mercurio

```

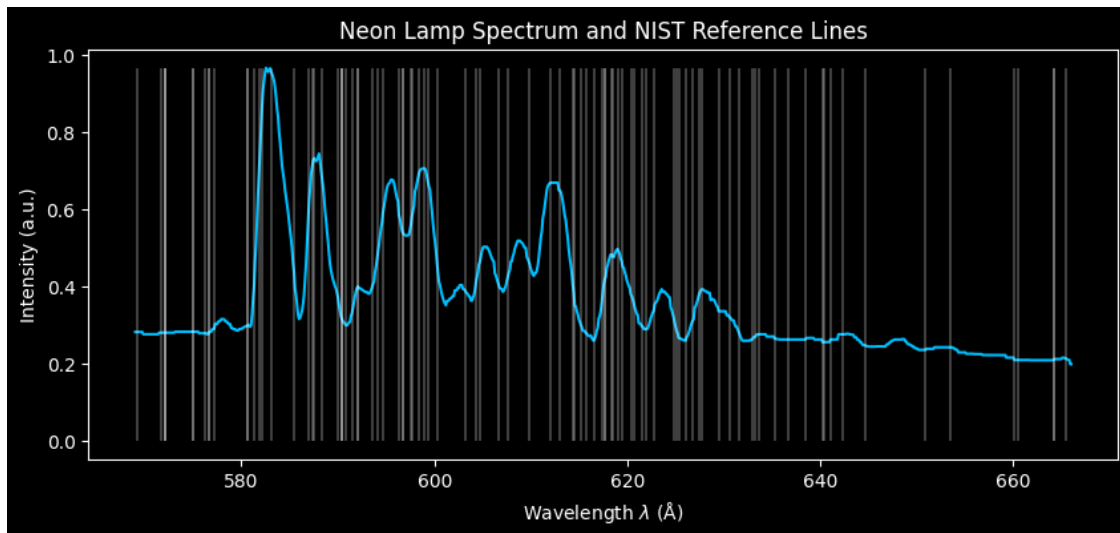
[251]: plt.figure(figsize=(10,4))
plt.plot(wavelengths_hg, hg_spectrum, color='deepskyblue')
plt.vlines(mercury_lines['Observed'], 0, max(hg_spectrum)*1,
          color='white', alpha=0.3, lw=1.2)
plt.xlabel(r"Wavelength  $\lambda$  (Å)")
plt.ylabel("Intensity (a.u.)")
plt.title("Mercury Lamp Spectrum and NIST Reference Lines")
plt.show()

```



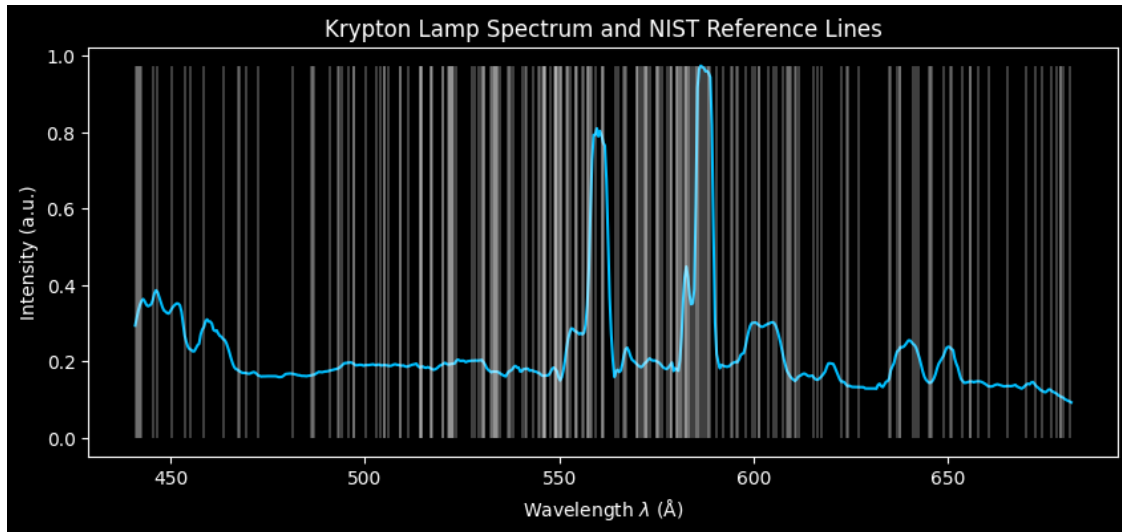
Neón

```
[252]: plt.figure(figsize=(10,4))
plt.plot(wavelengths_ne, ne_spectrum, color='deepskyblue')
plt.vlines(neon_lines['Observed'], 0, max(ne_spectrum)*1,
           color='white', alpha=0.3, lw=1.2)
plt.xlabel(r"Wavelength $\lambda$ (Å)")
plt.ylabel("Intensity (a.u.)")
plt.title("Neon Lamp Spectrum and NIST Reference Lines")
plt.show()
```



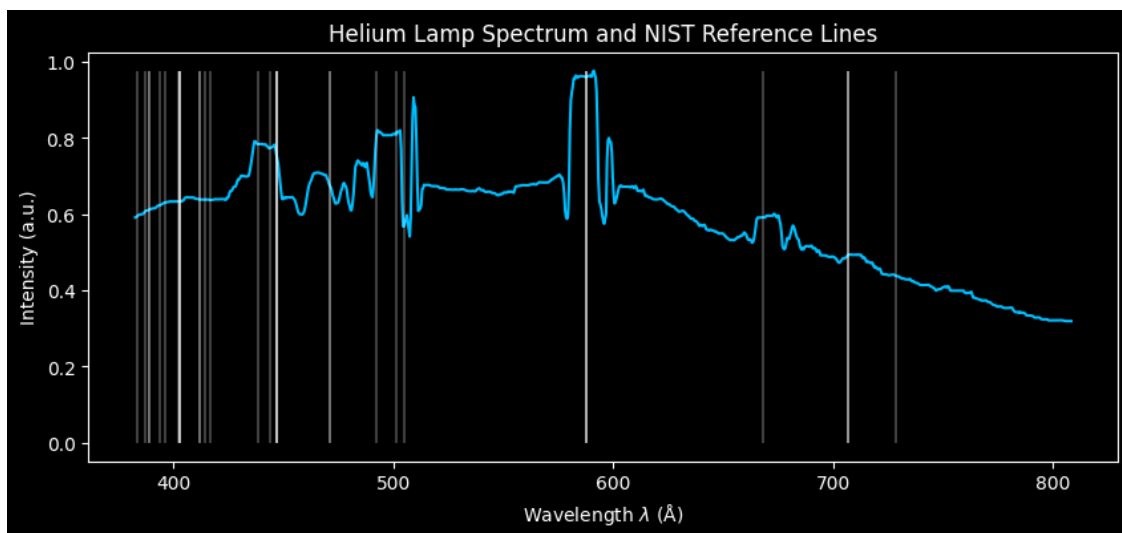
Kriptón

```
[253]: plt.figure(figsize=(10,4))
plt.plot(wavelengths_kr, kr_spectrum, color='deepskyblue')
plt.vlines(krypton_lines['Observed'], 0, max(kr_spectrum)*1,
           color='white', alpha=0.3, lw=1.2)
plt.xlabel(r"Wavelength $\lambda$ (Å)")
plt.ylabel("Intensity (a.u.)")
plt.title("Krypton Lamp Spectrum and NIST Reference Lines")
plt.show()
```



Helio

```
[254]: plt.figure(figsize=(10,4))
plt.plot(wavelengths_he, he_spectrum, color='deepskyblue')
plt.vlines(helium_lines['Observed'], 0, max(he_spectrum)*1,
          color='white', alpha=0.3, lw=1.2)
plt.xlabel(r"Wavelength  $\lambda$  (Å)")
plt.ylabel("Intensity (a.u.)")
plt.title("Helium Lamp Spectrum and NIST Reference Lines")
plt.show()
```



¡Has terminado! Ahora aplica esta calibración a los datos y comienza a medir cosas en el Tutorial

3.